

Fundamentals of API Development

API Contracts: Standards & OpenAPI/Swagger

Day 2 - Session 1

Day 1 Recap

From yesterday	Feeds today
API purpose, the contract asymmetry	Why a contract needs a standard shape
Audience types	Who reads your OpenAPI document
REST conventions, Lab 1.1 evidence	What Lab 1.2's spec should reflect

You brought findings from Lab 1.1. Today you stop analysing contracts and start writing one.

What We'll Cover

1. Why a contract needs a standard, machine-readable shape
2. Anatomy of an OpenAPI 3.1 document
3. Demo: validating a hand-written spec
4. Common mistakes that break the contract
5. Lab 1.2: writing an OpenAPI specification

Why Standards

A written contract only helps if machines can read it too

The Drawback of a Free-Form Description: Discussion

Imagine you wrote a beautifully detailed wiki page to describe your new API endpoint. A month later, the backend team adds a required field to the request body.

What are the chances the frontend team finds out before things break in production?

DISCUSS

Why do hand-written docs always drift?

The Drawback of a Free-Form Description: The Reality

A prose description of an API drifts, because nothing forces it to track the code, and every reader interprets it differently.

- No validator can check a paragraph
- No tool can render a paragraph into an interactive console
- No generator can build a client from a paragraph

A standard format fixes all three at once.

OpenAPI Is That Standard

OpenAPI 3.1 is the current stable baseline; 3.2, released September 2025, is the newest feature release on the same JSON-Schema-aligned foundation.

NOTE

Swagger 2.0 is an older, different format. Copying a Swagger 2.0 example into a 3.1 document is a common and confusing mistake.

OpenAPI Standards (Financial Services)

REAL-WORLD SCENARIO

Financial Services (Navy Federal Credit Union — API Exchange):

- NFCU's API Exchange publishes an open banking catalog of 8 data-access products (accounts, transactions, statements) secured behind OAuth member consent and mTLS.
- Every partner integration reads the same machine-readable contract. A free-form wiki could not enforce the authentication model or data shapes across 14M members.

OpenAPI Standards (Retail & E-Commerce)

REAL-WORLD SCENARIO

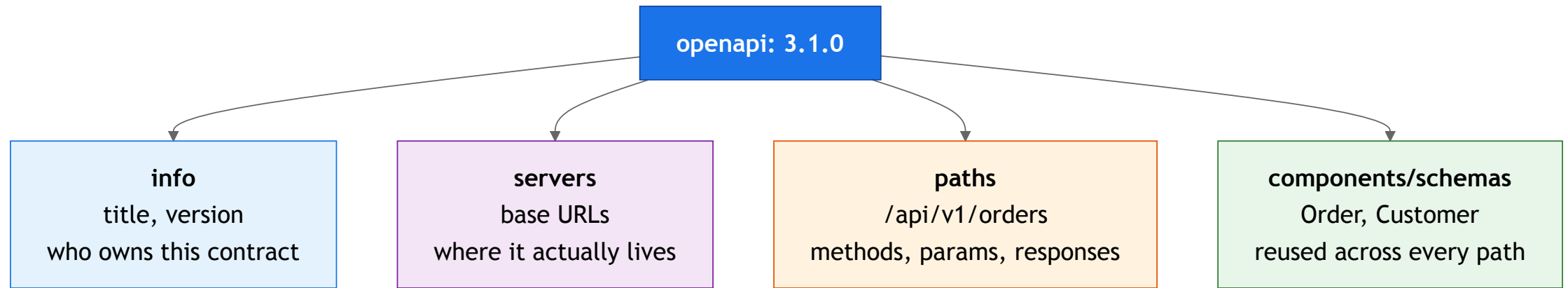
Retail & E-Commerce (Shopify — Admin REST API):

- Shopify publishes a complete OpenAPI 3.1 specification for its Admin REST API, covering products, customers, orders, inventory, and fulfillment.
- Third-party app developers generate client SDKs directly from the spec. No wiki page could serve the 2M+ merchants building integrations against that API surface.

Anatomy of a Spec

Four sections worth knowing cold

The Four Sections



Reading the Sections

Section	Holds
info	Title, version, contact
servers	The base URLs the API actually runs on
paths	Every endpoint: methods, parameters, responses
components/schemas	Reusable data shapes, referenced by <code>\$ref</code>

A schema defined once and referenced everywhere keeps naming consistent across the whole document.

API Contract Anatomy (Financial Services)

REAL-WORLD SCENARIO

Financial Services (Capital One — Microservice Catalog):

- Capital One requires every microservice to register its API contract in a central catalog, with info, servers, paths, and components/schemas defined before backend code exists.
- UI engineers build against the deployed contract within 48 hours. The schema section defines reusable account and transaction shapes referenced by \$ref across every service.

API Contract Anatomy (Retail & E-Commerce)

REAL-WORLD SCENARIO

Retail & E-Commerce (Walmart — Multi-Channel Solutions):

- Walmart's MCS API ships an OpenAPI spec that partners feed into OpenAPI Generator to produce Java clients. The components/schemas section defines reusable order, fulfillment, and inventory shapes.
- Every endpoint references those shapes by \$ref. When Walmart adds a field to the Order schema, every endpoint inherits it without a single path-level edit.

A Minimal Spec

REFERENCE ONLY

```
openapi: 3.1.0
info:
  title: Accounts API
  version: 1.0.0
paths:
  /accounts/{id}:
    get:
      summary: Get an account by id
      responses:
        '200':
          description: Account found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Account'
```

Demo: Validating a Hand-Written Spec

From `day2/demos`, run `java -cp target/classes com.kavinschool.demos.DemoOpenApiSpecValidator`.

- Loads a hand-written OpenAPI YAML file
- Checks for a valid `openapi` version field, a non-empty `info`, and every `$ref` resolving to a real schema
- Flags a spec that still says `swagger: "2.0"` instead of `openapi: "3.1.0"`

Offline, no API key, no network.

Where Contracts Break

The same three mistakes, every time

Three Recurring Mistakes

1. Writing the spec after the code, then letting the two drift apart
2. Leaving `description` fields blank throughout the document
3. Mixing a `swagger: "2.0"` root with 3.1-style syntax elsewhere

CAUTION

A spec that parses without error can still be useless to a consumer.

Contract Integrity (Financial Services)

REAL-WORLD SCENARIO

Financial Services (UK Open Banking — Conformance Suite):

- UK Open Banking's automated conformance suite runs contract tests against every bank's API. A spec that parses but drifts from the implementation fails certification.
- Banks must keep their OpenAPI spec in lockstep with the running code. The conformance tool flags mismatched fields, missing error responses, and stale schemas before a partner ever hits them.

Contract Integrity (Retail & E-Commerce)

REAL-WORLD SCENARIO

Retail & E-Commerce (Target — Partner API Program):

- Target's partner API program found that inline schema duplication across endpoints caused catalog and inventory fields to diverge silently between search, price, and availability responses.
- Moving every resource to centralized \$ref schemas eliminated field-level inconsistencies. A single schema definition now feeds every endpoint that returns a product.

Demo: Duplicated vs Referenced Schemas

From `day2/demos`, run `java -cp target/classes com.kavinschool.demos.DemoSchemaReuse`.

- Two versions of the same spec: one with `Account` duplicated inline in three paths, one with a single `$ref`
- A field is added to the account shape in only one place
- The duplicated version silently goes inconsistent; the `$ref` version cannot

Offline, no API key, no network.

Lab 1.2: Writing an OpenAPI Specification

Open `day2/lab1_2_writing_an_openapi_specification/` in the companion repo.

Goal: Write a complete OpenAPI 3.1 specification for a basic resource on the shared quickstart API.

1. Fill in `info` and `servers` for the quickstart API
2. Define at least one path with a described request and response
3. Define the resource once under `components/schemas` and reference it with `$ref`

Stretch: add request/response examples and an error schema for a non-2xx response.

Key Takeaways

1. A contract only works as a contract if it is written in a format machines can also read.
2. OpenAPI 3.1 organizes a spec into info, servers, paths, and reusable components/schemas.
3. A schema referenced by `$ref` cannot drift the way a duplicated inline schema can.
4. Writing the spec after the code is the single most common way contracts stop being trustworthy.

Questions?

Next: Kahoot 1, then a bio break